

Laborator 1.

Arhitecturi integrate de calculatoare de timp real. Microcontrolerul AVR ATMEGA128.

Medii de dezvoltare software pentru aplicații de timp real. IAR pentru AVR.

1. Arhitecturi de calculatoare de timp real. AVR ATMEGA128

Între un calculator de timp real și unul pentru aplicații uzuale nu există deosebiri esențiale. În esență un sistem de timp real trebuie să asigure un răspuns imediat la evenimente externe. Acesta presupune disponibilitatea pentru captarea evenimentului și tratarea răspunsului la acesta în orice moment al programului.

Deci, ceea ce deosebește un sistem de timp real de unul obișnuit este modul în care este organizat programul, astfel încât, având un singur procesor care poate opera la un anumit moment, să poată totuși îndeplini o cerință tipică mai degrabă unor sisteme paralele. Aceasta se poate realiza prin descompunerea sarcinilor care trebuie să le asigure sistemul și repartizarea acestora pe procese care se execută concurrent.

În aceste condiții, singura cerință hardware impusă pentru a putea realiza un sistem de timp real este ca să fie asigurate resursele pentru a putea implementa procesele concurente.

În figura 1 este prezentată o arhitectură tipică de calculator de timp real:

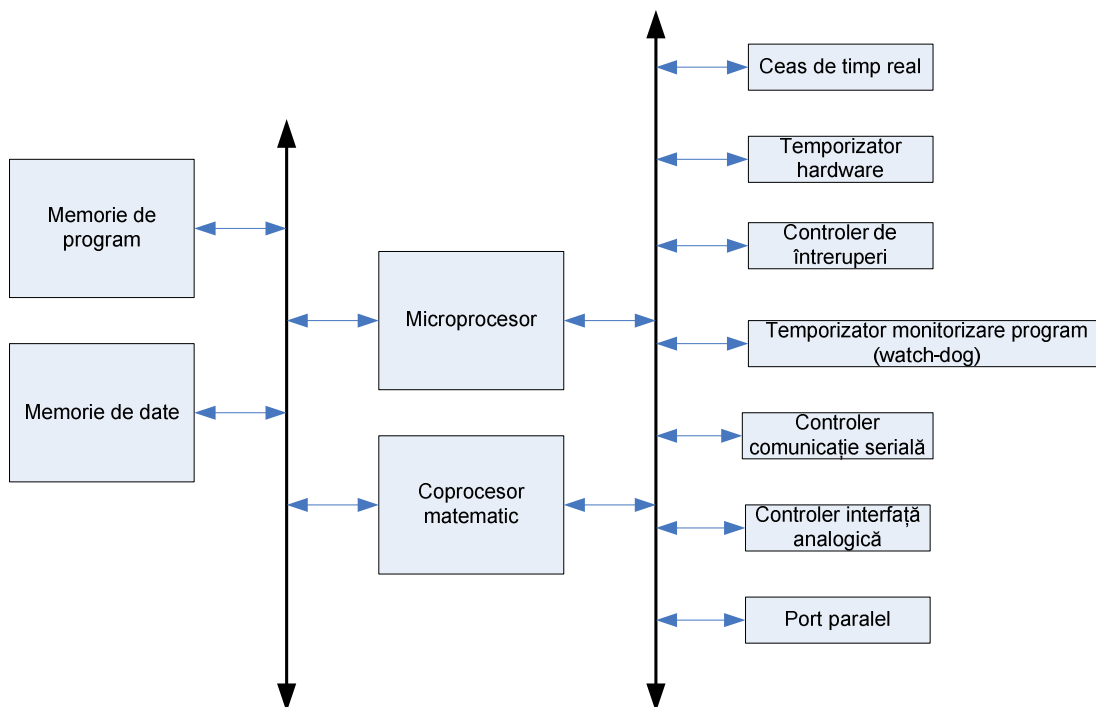


Fig.1. Schema bloc al unui calculator de timp real

Când vorbim de resurse necesare pentru un STR ne referim la:

- memorie de program pentru implementarea tehnicilor de programare concurentă (administrarea proceselor, comutare context, etc);
- memorie de date pentru lucrul cu variabile contor, semafoare și pentru salvarea temporală a contextului;
- circuite periferice pentru captarea și contorizarea timpului, necesare pentru contorizarea proceselor, a evenimentelor, sincronizare comunicații;
- alte circuite periferice care permit comunicarea cu procesele externe controlate – interfețe paralele, seriale, analogice – în funcție de specificul aplicației.

ATmega128 este un microcontroler pe 8 biți realizat cu o arhitectură avansată de tip RISC. Are capacitatea de executare a mai multor instrucțiuni într-un singur ciclu de ceas, ATmega128 poate realiza cu aproximație 1 MIPS / MHz, ținându-se cont la proiectarea sistemului de optimizarea consumului energetic în raport cu viteza de procesare.

Nucleul microcontrolerului AVR îl constituie un set bogat de instrucțiuni alături de cei 32 de regiștri de lucru cu scop general. Toți cei 32 de regiștri sunt conectați la Unitatea Aritmetică și Logică (ALU), acest lucru permite accesarea a doi regiștri independenți într-o singură instrucțiune executată într-un ciclu mașină. Arhitectura rezultată este mult mai eficient codată putând realiza viteze de zece ori mai mari decât un microcontroler convențional de tip CISC.

Atmega128 conține următoarele structuri de memorie: 128 Kb în sistem de programare rapidă alături de capacitatea de a citi și scrie simultan, 4 Kb de memorie SRAM, 32 de regiștri de lucru cu scop general.

Din punct de vedere al resurselor I/O amintim 53 de linii tip port paralel I/O, un numărător în timp real (RTC), patru numărătoare/ temporizatoare cu mod de comparare și PWM, 2 USART-uri, o interfață serială pe două fire, 8 canale analogice de intrare, respectiv diferențiale cu câștig variabil, un convertor analog-digital pe 10 biți, numărătorul pentru circuitul Watchdog programabil cu ajutorul oscilatorului intern, un port serial SPI.

În cele ce urmează vom face o trecere în revistă a circuitelor periferice insistând mai mult pe aspectele legate de configurarea acestora în vederea executării anumitor programe. Vor fi prezentate doar circuitele periferice care vor fi folosite în laboratoarele care urmează, pentru detalii se va consulta foaia de catalog a microcontroler-ului.

Porturile paralele

Microcontrerul ATMEGA128 are 7 porturi paralele notate cu litere A,B,..G. Dintre acestea 6 sunt pe 8 biți (A..F) și unul pe 5 biți (G). Toate porturile se pot configura ca intrări sau ieșiri la nivel de bit.

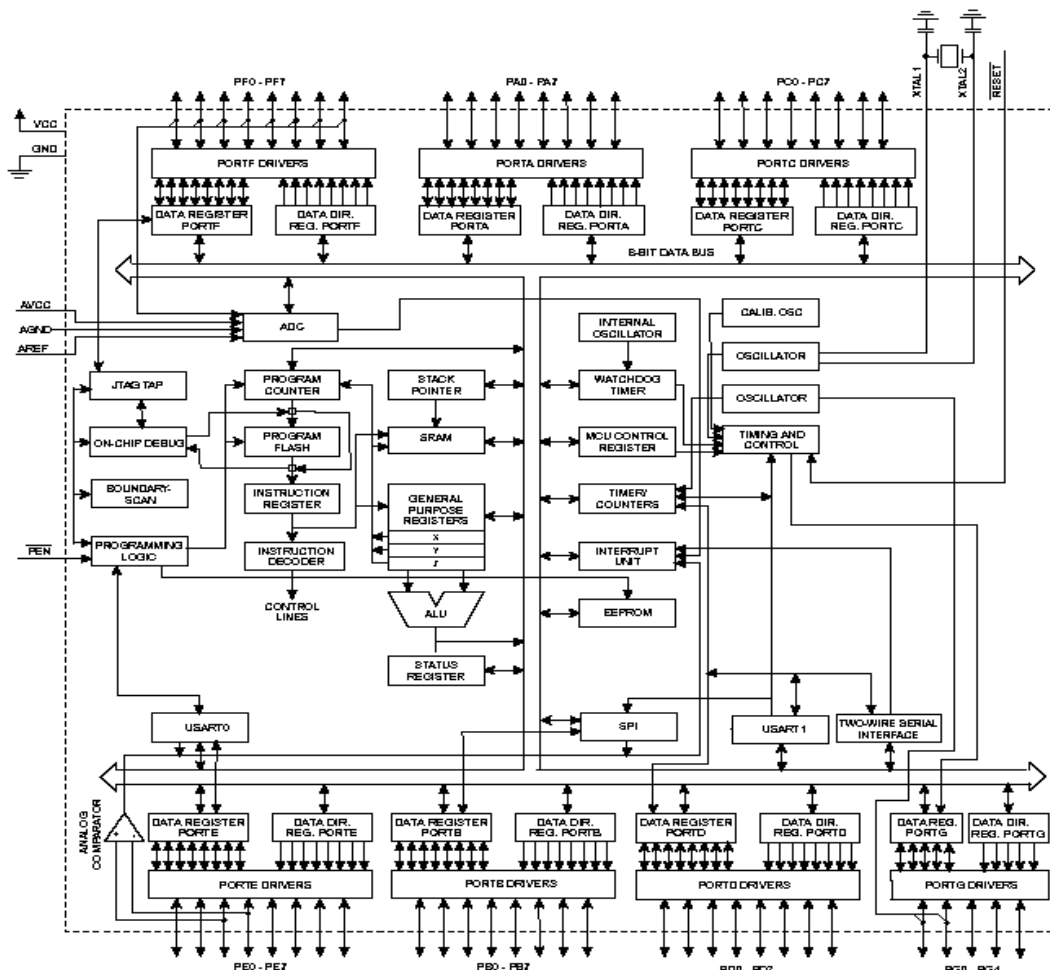


Fig.2. Schema bloc AVR ATmega128 (foaie de catalog)

Configurarea direcționalității unui port paralel se face scriind în registrul DDRx (x corespunde unui port A..G) cu următoarea organizare:

DDR_x

IO7	IO6	IO5	IO4	IO3	IO2	IO1	IO0
-----	-----	-----	-----	-----	-----	-----	-----

La fiecare bit din registru îi corespunde un bit de pe portul paralel direcționat astfel:

IO_i = 1 => bitul i al portului va fi configurat ca ieșire,

IO_i = 0 => bitul i al portului va fi configurat ca intrare.

Scrierea pe un port, dacă acesta a fost configurat ca ieșire, se face prin scrierea în registrul PORTx (x: A..G) iar citirea de pe un port, dacă acesta a fost configurat ca port de intrare se face prin citirea din registrul PINx. Organizarea regiștrilor PORTx și PINx este similară. PORTx este accesibil numai pentru scriere iar PINx numai pentru citire.

PORTx

WR7	WR6	WR5	WR4	WR3	WR2	WR1	WR0
-----	-----	-----	-----	-----	-----	-----	-----

PINx

RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0
-----	-----	-----	-----	-----	-----	-----	-----

Fiecare pin de port paralel are o funcție alternativă pe care o poate executa în condițiile în care se dorește acest lucru. În tabelele de mai jos sunt prezentate funcțiile alternative pentru toate porturile ATmega.

Pin port	Funcție alternativă
PORTA	
PA7	AD7 (adresă/dată externă)
PA6	AD6 (adresă/dată externă)
PA5	AD5 (adresă/dată externă)
PA4	AD4 (adresă/dată externă)
PA3	AD3 (adresă/dată externă)
PA2	AD2 (adresă/dată externă)
PA1	AD1 (adresă/dată externă)
PA0	AD0 (adresă/dată externă)
PORTB	
PB7	OC2/OC1C (ieșire timer 2 sau ieșire canal C timer 1)
PB6	OC1B (ieșire timer 1 canal B)
PB5	OC1A (ieșire timer 1 canal A)
PB4	OC0 (ieșire timer 0)
PB3	MISO (interfață SPI – comunicație serială sincronă)
PB2	MOSI (interfață SPI – comunicație serială sincronă)
PB1	SCK (interfață SPI – comunicație serială sincronă)
PB0	SS (interfață SPI – comunicație serială sincronă)
PORTC	
PC7	A15(adresă externă)
PC6	A14(adresă externă)
PC5	A13(adresă externă)
PC4	A12(adresă externă)
PC3	A11(adresă externă)
PC2	A10(adresă externă)
PC1	A9(adresă externă)
PC0	A8(adresă externă)
PORTD	
PD7	T2 (intrare ceas timer 2)
PD6	T1 (intrare ceas timer 1)
PD5	XCK1 (intrare ceas comunicație serială asincronă canal 1)
PD4	IC1 (intrare captură canal 1)
PD3	INT3/TXD1 (întrerupere externă 3 sau transmisie serială canal 1)
PD2	INT2/RXD1 (întrerupere externă 2 sau recepție serială canal 1)

PD1	INT1/SDA (întrerupere externă 1 sau comunicație I2C serială sincronă)
PD0	INT0/SCL (întrerupere externă 0 sau comunicație I2C serială sincronă)
PORTE	
PE7	INT7/IC3 (întrerupere externă 7 sau intrare captură timer 3)
PE6	INT6/T3 (întrerupere externă 6 sau intrare ceas extern timer 3)
PE5	INT5/OC3C (întrerupere externă 5 sau ieșire canal C timer 3)
PE4	INT4/OC3B (întrerupere externă 5 sau ieșire canal B timer 3)
PE3	AIN1/OC3A (intrare analogică 1 sau ieșire canal A timer 3)
PE2	AIN0/XCK0 (intrare referință analogică 0 sau intrare ceas comunicație serială asincronă canal 0)
PE1	PDO/TXD0 (ieșire programare sau transmisie comunicație serială asincronă canal 0)
PE0	PDI/RXD0 (intrare programare sau recepție comunicație serială asincronă canal 0)
PORTF	
PF7	ADC7/TDI (interfață analog-digitală sau interfață JTAG)
PF6	ADC6/TDO (interfață analog-digitală sau interfață JTAG)
PF5	ADC5/TMS (interfață analog-digitală sau interfață JTAG)
PF4	ADC4/TCK (interfață analog-digitală sau interfață JTAG)
PF3	ADC3 (interfață analog-digitală)
PF2	ADC2 (interfață analog-digitală)
PF1	ADC1 (interfață analog-digitală)
PF0	ADC0 (interfață analog-digitală)
PORTG	
PG4	TOSC1 (intrare oscilator timp real)
PG3	TOSC2 (intrare oscilator timp real)
PG2	ALE (comandă magistrală externă)
PG1	RD (comandă magistrală externă)
PG0	WR (comandă magistrală externă)

Dacă se lucrează în oricare din aceste funcții alternative în mod evident pinul alocat nu mai poate fi folosit și ca pin general IO.

Configurarea unui port paralel se face în următorii pași:

1. se setează direcționalitatea liniilor portului (intrări sau ieșiri);
2. se efectuează pentru liniile de ieșire inițializarea lor prin scrierea valorilor de pornire.

Circuitele temporizatoare-numărătoare

ATMEGA are 4 circuite temporizatoare-numărătoare: două circuite pe 8 biți (numărător/temporizator 0 și numărător/temporizator 2) și alte două pe 16 biți

(numărător/temporizator 1 și numărător/temporizator 3). Circuitele pe 16 biți au 3 canale independente pe care se poate face captura semnalelor externe, adică pot număra 3 semnale externe distincte.

Toate circuitele lucrează în mai multe moduri. Ele primesc un semnal de ceas, extern sau intern, în ritmul căruia numără, în sens crescător. Intervalul de numărare maxim este pentru cele pe 8 biți între 00 și FFh (256) iar pentru cele pe 16 biți între 0000h și FFFFh (65535). Circuitul semnalizează momentul în care a depășit valoarea maximă, în modul numărător sau când a atins o valoare aflată în registrul OCRx în modul comparator. Valoarea curentă a numărătorului (temporizatorului) se află stocată în regiștrii TCNT0 și TCNT2 (pe 8 biți) și în regiștrii TCNT1 și TCNT3 (pe 16 biți).

Alături de constanta de divizare care se configurează scriind registrul TCNTx circuitele numărător mai conțin și în plus și un prescaler. Acesta poate realiza divizări ale ceasului cu: 1, 8, 64, 256, 1024.

Circuitele de numărare au mai multe moduri de lucru:

- numărător normal (normal) – divizor de frecvență: va număra în sens crescător pornind de la valoarea scrisă în registrul TCNT. Va semnaliza momentul în care a ajuns la maxim (FFh sau FFFFh). În acest moment numărătoarea se va opri. Reluarea numărării se va face prin reîncărcarea registrului TCNT. Circuitul poate genera cerere de întrerupere în momentul în care a ajuns la maxim;
- comparare cu o valoare (clear timer on compare match): va număra în sens crescător pornind de la valoarea scrisă în TCNT. Va semnaliza momentul în care starea curentă a numărătorului este egală cu valoarea care există în registrul OCR. Automat resetează valoarea lui TCNT deci se va relua numărătoarea. Ieșirea corespunzătoare circuitului sau canalului (OCi) va fi setată într-o stare opusă celei anterioare la fiecare sfârșit al numărării. Circuitul poate genera cerere de întrerupere când se ajunge la egalitate în urma comparării;
- modulator impulsuri în lățime (fast PWM): va număra în sens crescător pornind de la valoarea scrisă în TCNT. Va semnaliza momentul în care a ajuns la o valoare egală cu cea aflată în registrul OCR. În acest moment ieșirea OCi va fi resetată (pusă în 0). Se va continua numărarea până când va ajunge la valoarea maximă. Este semnalizată și atingerea acestei valori iar ieșirea OCi este pusă în 1. Se reia numărarea. Circuitul poate genera cerere de întrerupere când se ajunge la egalitate cu OCR și/sau când este atinsă valoarea maximă;
- modulator impulsuri în lățime defazat (phase correct PWM): va număra în sens crescător. La fel ca și în modul anterior, când ajunge la o valoare egală cu cea din registrul OCR resetează ieșirea OCi. Circuitul continuă numărarea. Semnalizează în momentul în care ajunge la valoarea maximă. Din acest moment reia numărarea în sens descrescător. Când ajunge din nou la valoarea lui OCR setează ieșirea OCi în 1. Continuă numărarea descrescătoare până la 0 după care reia. Circuitul poate genera cerere de întrerupere când este atinsă valoarea din OCR atât pe frontul crescător cât și pe cel descrescător.

Pentru operarea cu canale timer se va proceda astfel:

1. se stabilește modul de lucru;
2. se stabilește sursa ceasului (temporizator sau numărător);

3. se stabilește valoarea registrului OCR dacă se lucrează în modul comparator, modulator PWM sau modulator PWM defazat și a registrului TCNT dacă se lucrează în modul normal;
4. se activează lucrul cu întreruperi dacă se dorește.

Circuite seriale

ATMEGA are 2 canale seriale UART0 și UART1. Acestea pot lucra independent fiecare putând avea următoarele caracteristici:

- ceas generat intern sau extern (pe pinii XCK0 sau XCK1) pentru stabilirea ratei de transfer;
- pot asigura transmisie și (sau) recepție serială asincronă și sincronă;
- pachetul poate conține 5,6,7,8 sau 9 biți de date;
- pachetul poate avea sau nu bit de paritate;
- pachetul poate conține 1 sau 2 biți de stop;
- viteza de comunicație se poate seta prin configurarea regiștrilor UBRLx și UBRRHx (care concatenați constituie registrul pe 16 biți UBRRx) corespunzători fiecărui canal astfel:

Mod de operare	Calcul viteză de comunicație (bps) în funcție de UBRR	Calcul valoare UBRR în funcție de viteza de comunicație
Asincron, mod normal (U2X = 0)	$\text{Viteza} = \frac{f_{\text{osc}}}{16(\text{UBRR} + 1)}$	$\text{UBRR} = \frac{f_{\text{osc}}}{16 \text{ Viteza}} - 1$
Asincron, mod viteză dublă (U2X = 1)	$\text{Viteza} = \frac{f_{\text{osc}}}{8(\text{UBRR} + 1)}$	$\text{UBRR} = \frac{f_{\text{osc}}}{8 \text{ Viteza}} - 1$
Sincron, mod master	$\text{Viteza} = \frac{f_{\text{osc}}}{2(\text{UBRR} + 1)}$	$\text{UBRR} = \frac{f_{\text{osc}}}{2 \text{ Viteza}} - 1$

- întreruperea poate apărea la recepție sau la emisia unui octet.

Pentru lucrul cu circuitele seriale se procedează astfel:

1. Se stabilește frecvența de comunicație și se calculează valoarea din registrul UBRR;
2. Se stabilește numărul de biți de date pe un pachet;
3. Se stabilește numărul de biți de paritate și numărul de biți de stop;
4. Se activează întreruperile, dacă se dorește lucrul cu întreruperi.

Medii de dezvoltare software pentru aplicații de timp real. Pachetul IAR

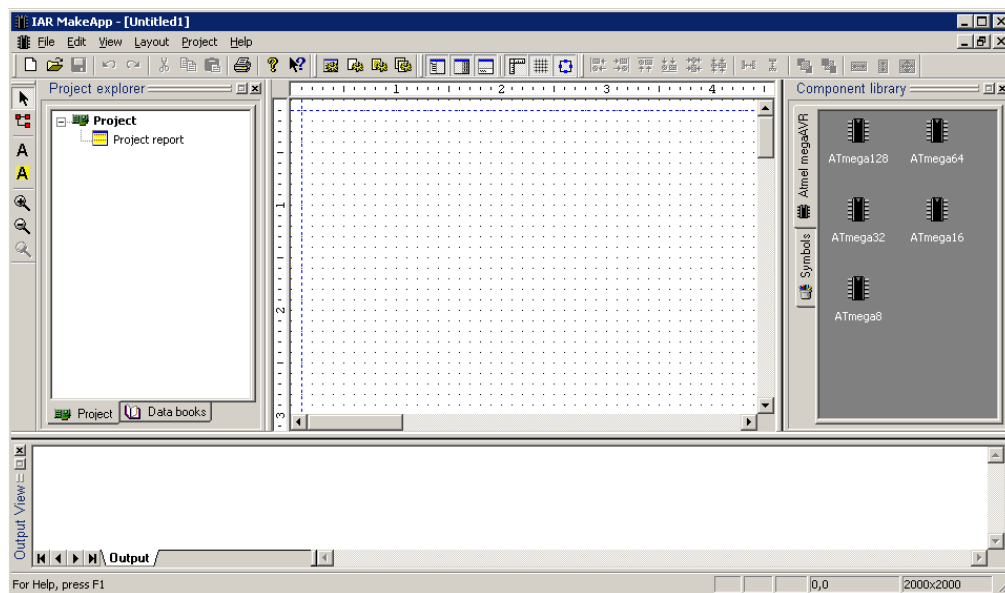
IAR Make Application

Este utilizată pentru configurarea facilă a microcontrolerului AVR ATmega. Acesta implică atât nucleul de procesare cât și circuitele periferice.

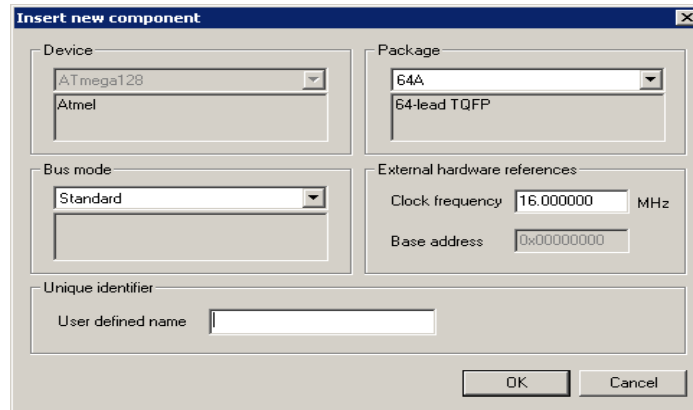
Sunt introduse opțiunile referitoare la configurația circuitelor perifericelor. Se obține un set de librării descrise în limbajul C (fișiere .c și .h).

Modul de utilizare este următorul:

1. se rulează aplicația MakeApplication (shortcut pe desktop);
2. primul ecran al aplicației este unul cu un proiect gol deja deschis ca în figură;

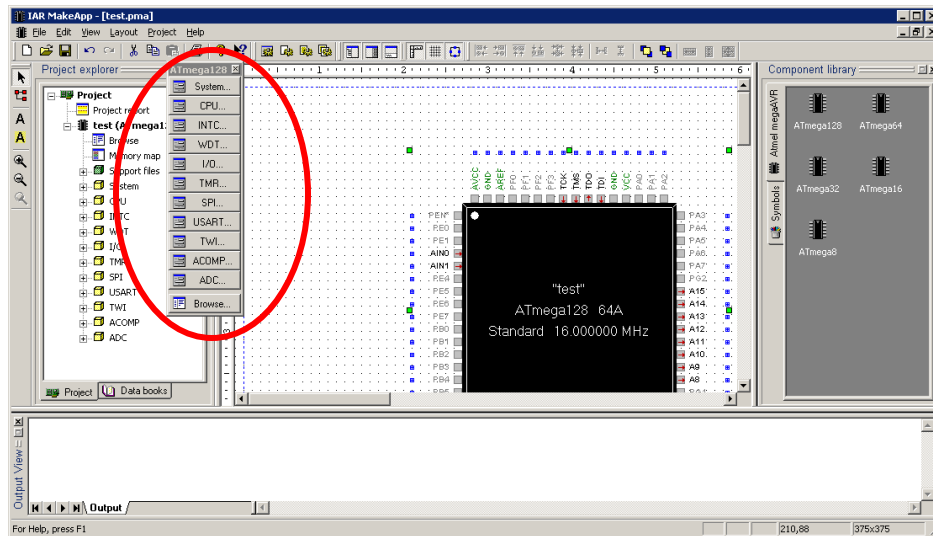


3. Se salvează proiectul (director pe discul D);
4. Din lista cu microcontrolere din stânga se „trage” folosind mouse-ul ATmega128;
5. Dacă manevra a fost executată corect va apărea afișată pe ecran fereastra de mai jos:



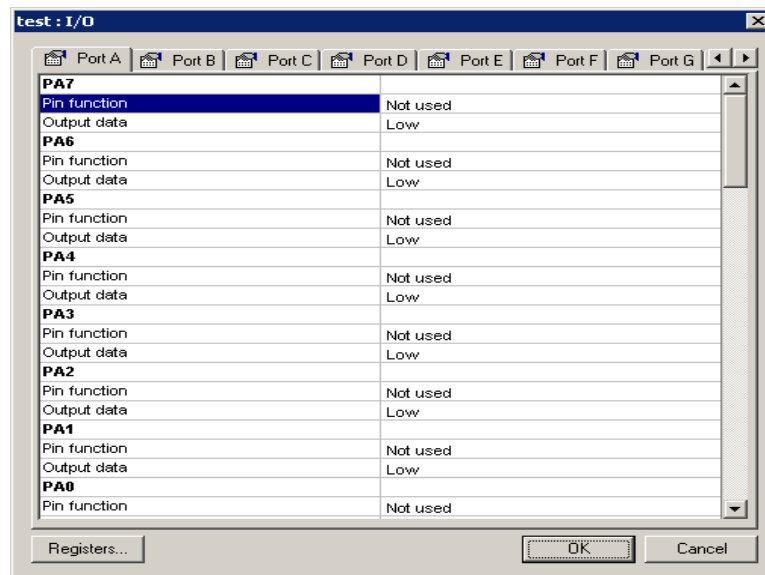
În câmpul editabil „User defined name” se introduce numele configurației (opțional, poate fi și numele proiectului) și se apasă apoi „OK”;

6. Fereastra proiectului ca trebui să arate acum ca în figura de mai jos:



Se observă în ecranul principal circuitul cu numele configurației pe el introdus și o listă cu butoane care reprezintă tot ce se poate configura la microcontroler (înconjurată în figură). Pentru exemplificare se apasă butonul I/O din listă ceea ce semnifică o configurare a porturilor paralele.

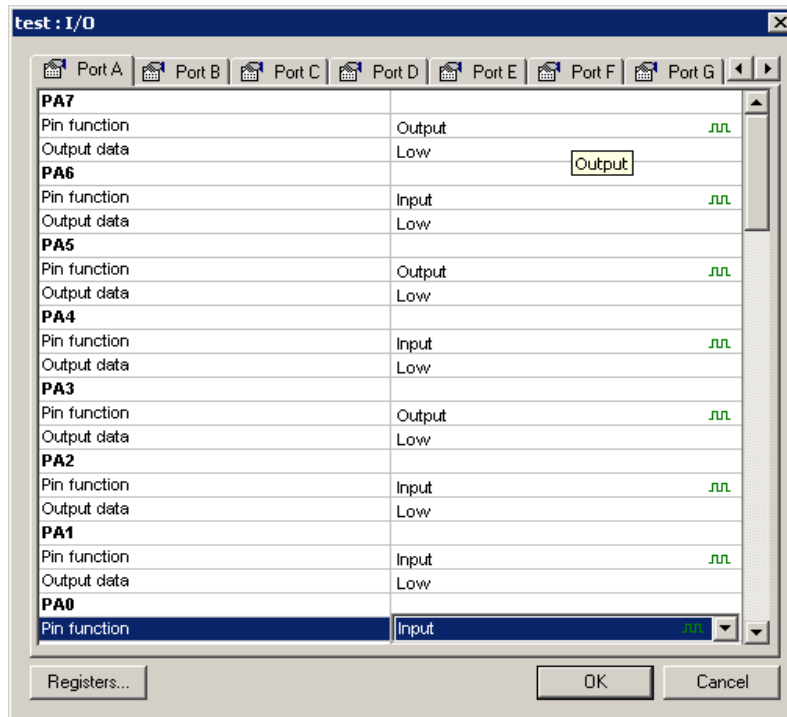
7. Va apărea fereastra pentru configurarea porturilor I/O ca în figura de mai jos:



Se observa în fereastră câte un TAG pentru fiecare port (A..G).

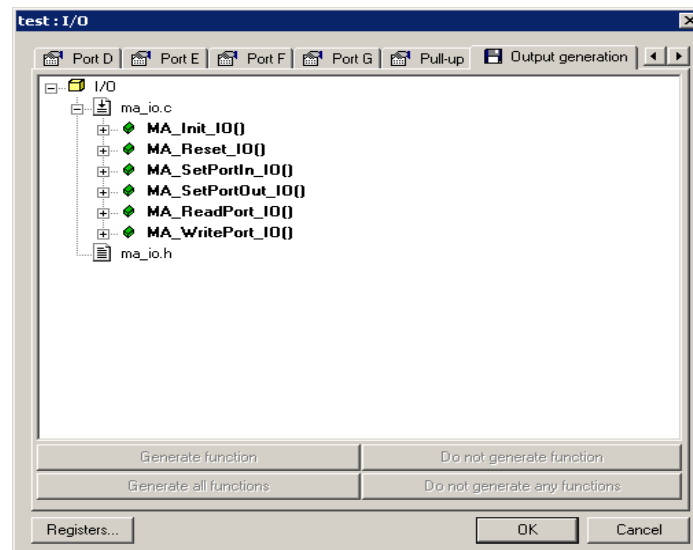
Conform celor menționate la prezentarea porturilor paralele, inițializarea lor se face în doi pași: setarea direcției și configurarea valorii inițiale pentru pinul respectiv. Se poate observa ca la fiecare pin, de pe fiecare port, îi corespunde două setări: „Pin function” care

se referă la direcționalitatea pinului și „Output data” care se referă la valoare (este luată în considerare numai dacă pinul a fost configurat ca ieșire). Configurarea se face accesând lista derulantă corespunzătoare liniei fiecărei setări (click mouse pe valoarea fiecărei setări). Pentru exemplificare se vor configura la portul A pinii 7,5,3 ca ieșiri și pinii 6,4,2,1,0 ca intrări. Fereastra va arăta ca mai jos:



„Output data” pentru ieșiri a fost lăsată setată cu valoarea „low” adică vor avea inițial valoarea 0.

8. Se selectează ultimul TAG din fereastră numit „Output generation”. Vom avea o fereastră ca mai jos:



În această fereastră sunt reprezentate toate funcțiile care vor fi generate și cele două fișiere numite `ma_io.c` și `ma_io.h`. Funcțiile au următoarele semnificații:

`MA_Init_IO` – inițializează porturile I/O conform setărilor care au fost realizate;

`MA_Reset_IO` – re setează porturile I/O la setările inițiale (toți pinii sunt neutilizați);

`MA_SetPortIn` – configurează pini din port ca intrare;

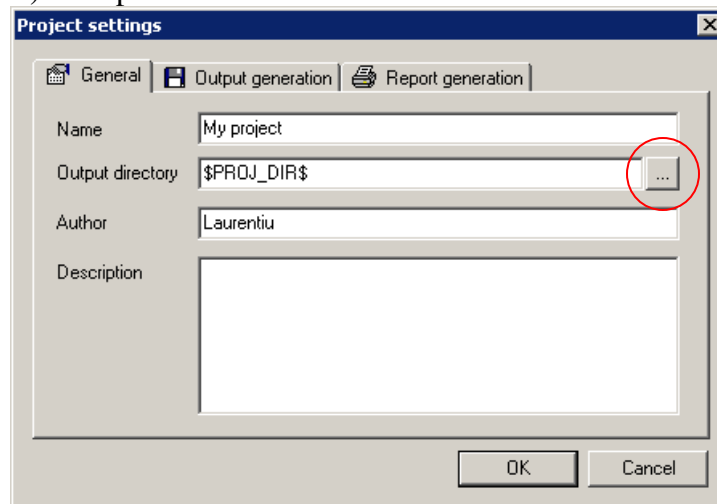
`MA_SetPortOut_IO` – configurează pini din port ca ieșire;

`MA_ReadPort_IO` – citește pini din port (pinii au fost configurați ca intrări);

`MA_WritePort_IO` – scrie pini din port (are efect numai dacă pinii au fost configurați ca ieșiri).

Se dă click pe numele fișierului `ma_io.c` și apoi se apasă butonul „Generate all functions” ceea ce înseamnă că toate funcțiile de mai sus vor fi generate ca funcții.

9. După ce setările au fost realizate se apasă OK pentru a fi închisă fereastra de setări pentru porturile IO. Pașii 6,7 și 8 se repetă și pentru setarea altor componente. După ce toate setările au fost realizate se selectează din meniul Project opțiunea Project Settings (sau se apasă butonul Project Settings din bara cu butoane). Va apărea următoarea fereastră:



În dreptul câmpului „Output directory” se apasă butonul de navigare (etichetat cu 3 puncte) și se selectează directorul unde a fost salvat și proiectul. Se apasă OK.

10. Se selectează din meniul Project opțiunea „Generate all” (sau se apasă butonul „Generate output and project report”). Este verificată corectitudinea setărilor și sunt generate librăriile care vor putea fi apoi importate într-un proiect, pentru construirea unei aplicații. Fișierele pot fi deschise cu un editor text și se poate observa codul din acestea, precum și explicațiile date la fiecare funcție, în limba engleză. De asemenea este construit un raport referitor la funcțiile realizate în fișierul `report.html`, aflat chiar în directorul unde au fost create librăriile.

Probleme.

1. Să se consulte fișierele `ma_io.c` și `ma_io.h` generate după urmărirea etapelor de mai sus. Să se determine parametrii și valorile returnate de funcții.
2. Urmărind pașii prezentați să se realizeze fișierele librărie cu următoarele configurații:

Portul A: 7,6,5,4 : intrări, 3,2,1,0 : ieșiri. Portul B: 7,6,3,2, : ieșiri 5,4,1,0 intrări.

3. Urmărind etapele prezentate la configurarea circuitelor temporizatoare să se configureze circuitul temporizator, cu sursa ceasului internă, în modul normal de lucru și un prescaler de 256. Se vor consulta librăriile și se vor observa funcțiile. Ce se schimbă în librărie la activarea întreruperii?
4. Să se configureze circuitul UART0 să poată recepționa și transmite pachete de date pe 8 biți, 1 bit de stop fără paritate. Rata de transfer a datelor dorită este 9600 bps. Frecvența de lucru a sistemului este de 16MHz.